

frplib Cheatsheet

C. R. Genovese

04 Oct 2026 v0.3.10 (frplib v0.3.10)

frplib is an educational platform for probabilistic computation. It can be used either as a Python library or as an interactive tool.

Below, names starting with **kX** denote Kinds/FRPs, **k** Kinds, **X** FRPs, **ckX** conditional Kinds/conditional FRPs, **ck** conditional Kinds, **cX** conditional FRPs, **stat** or **s** statistics, **cond** conditions, **n** natural numbers, **fn** or **f** functions, **v** values, and **obj** or **x** generic objects.

The Playground

An enhanced and configurable Python environment that is preloaded with **frplib** functions and objects.

Multi-line input is accepted and can be edited; a blank line at the end will enter the input.

Navigation. Arrows (or Control-n/p) move through your history; Control-a (beginning of line), Control-e (end of line), Control-k delete rest of line, F3 see full history, F2 settings menu, Control-r search history for pattern (type pattern and hit enter to select).

Help. The playground has a built-in help system.

info(), **info(obj)**, **info(topic)**

Interactive search of builtin documentation, or
see documentation for a specific object or topic.

help(obj, True) :: look at Python's builtin help for an object

explain_error() :: get more detail about a raised exception

cheatsheet() :: view or save this cheatsheet as PDF

cookbook() :: view or save the **frplib** cookbook as PDF

textbook() :: view or save the textbook as PDF

source(obj) :: view source code of function or callable object

Environment and Configuration. The **environment** object lets you query and set various features of the playground interface and computation. See **info(environment)** in the playground.

Quick Scripts. For quick Python scripts, you can load the playground from the **frplib.playground** module.

FRPs

A device that represents a random quantity whose value may not yet exist or be available

frp(kX) :: creates an FRP from a Kind or another FRP.

clone(X) :: a fresh, independent FRP with the same Kind as X

X.is_fresh :: tests whether FRP X is fresh

X.value :: X's value, activating it if necessary

conditional_frp(dict_or_fn)
make a conditional FRP from a dict or function
can be used as a function decorator

@frp_factory :: decorator for custom FRP factory

FRP.sample(n, frp_or_kind, summary=True)
generates n samples from **frp_or_kind** clones,
producing summary table (default) or all values

cX(v) or **cX.target(v)** :: target FRP for v

cX.joined(v) :: joined FRP for v

Kinds

A tree that captures all our knowledge/predictions about an FRP. Has values at each node and weights on each edge. Satisfies the Completeness, Level, Prefix, and Positivity Rules.

kind(obj) :: returns/builds the Kind of/from an object

conditional_kind(dict_or_fn)
make a conditional Kind from a dict or function
can be used as a function decorator

@kind_factory :: decorator for custom Kind factory

k.kernel(v) :: kernel function of a Kind k at v

k.entropy :: entropy of a Kind k

for val, wgt in k:
... # iterate over a Kind

Kind.equal(k1, k2), **Kind.compare(k1, k2)**
test/compare Kinds with specified tolerance (default: 1e-12)

Kind.divergence(k1, k2) :: Kullback-Leibler Divergence

ck(v) or **ck.target(v)** :: target Kind for v

ck.joined(v) :: joined Kind for v

ijoin_log_likelihood(v), **ijoin_log_kernel(k)**
return derived functions for independent join powers

Statistics

Functions from values to values. Embody questions about data.

@statistic

@statistic(opts...)

decorates a function to create a statistic

statistic(fn, opts...) :: creates a statistic from a function

@scalar_statistic

@scalar_statistic(opts...)

like **@statistic** but forces dimension 1

scalar_statistic(fn, opts...)
like **statistic** but forces dimension 1

@condition

@condition(opts...)

decorates a function to create a condition

condition(fn, opts...)

creates a condition from a function

stat(v) :: evaluate stat at **as_quant_vec(v)**

stat(x1, x2, ..., xn)

evaluate stat at $\langle x1, x2, \dots, xn \rangle$

v ^ stat :: evaluate stat at vector tuple v

@statistic_factory :: decorator for custom statistic factory

@condition_factory :: decorator for custom condition factory

The Big 3+1

Transforming with Statistics

kX ^ stat

stat(kX)

stat @ kX :: like **stat(kX)** but keeps context with observations

@frp	@kind
def X():	def k():
v = yield kX	v = yield k0
return stat(v)	return stat(v)

ckX ^ stat or **ckX.transform_joined(stat)**
transform joined outputs to new conditional Kind/FRP

ckX.transform_joined(stat)
transform targets to new conditional Kind/FRP

Building with Joins

kX1 * kX2 :: independent join of kX1 and kX2

kX ** n :: independent join power, clones for FRPs

kX >> ckX :: general join producing a Kind or FRP

@frp	@kind
def X():	def k():
u = yield kX	u = yield k0
v = yield ckX(u)	v = yield ck(u)
return (u, v)	return (u, v)

ckX // kX :: conditioning – join then projection

@frp	@kind
def X():	def k():
u = yield kX	u = yield k0
v = yield ckX(u)	v = yield ck(u)
return v	return v

ckX1 * ckX2
conditional Kind/FRP with independent join of targets

ckX1 >> ckX2 :: general join producing conditional Kind/FRP

Updating with Observations

kX | cond - updates a Kind/FRP with an observation

psi@k | cond :: update a transform with original context

@frp	@kind
def X():	def k():
v = yield kX	v = yield k0
yield guard(cond(v))	yield guard(cond(v))
return v	return v

Predicting with Expectations

E(kX) :: the expectation of a Kind or FRP

E(ckX) :: statistic mapping inputs to target expectations

Var(kX) :: the variance of a Kind or FRP

D_(kX) :: distribution operator, $D_-(X)(stat)$ gives $E(stat(X))$

FRP Factories

shuffle(coll)
FRP representing a random permutation of collection **coll**

Kind Factories

A **values...** argument can be multiple values as individual arguments, or any iterable, or an implied sequence of the form **a, b, ..., c** in a positive or negative direction. Values can be numbers or symbols or tuples thereof.

Kind.empty :: the empty Kind

constant(v) :: value **v** with weight 1

binary(p) :: values 0 and 1 with weights $1 - p$ and p

choice(a, b, r) :: values **a, b** with weights **1, r**

uniform(values...) :: specified values with equal weights

weighted_as(values..., weights=[...])

weighted_as({value: weight, ...})
specified values and corresponding weights

weighted_by(values..., fn)
specified values with **weight = fn(value)**

weighted_pairs((value, weight), ...)
specified values with their weights

arbitrary(values..., names=[...])
specified values and symbolic (unspecified) weights,
optional names assigned to corresponding weights

evenly_spaced(start, stop, num, by, wfn)
values are evenly spaced numbers with **weight = wfn(value)**

integers(start, stop, step, wfn)
values are integers **start + i · step**,
weights are **weight = wfn(value)**

geometric(values..., w_1, r)
*j*th value specified has weight $w_1 r^{j-1}$

linear(values..., first=a, increment=b)
values with weights varying linearly $a + b(v - v_1)$

permutations_of(values...)
permutations of values, equally weighted

subsets(coll)
values are subsets of collection **coll**, equally weighted

symmetric(values..., around, weight_by)
values weighted by a symmetric function **weight_by**
centered on **around**

without_replacement(n, values...)
ordered_sample(n, values...)
samples of *n* values, equally weighted

Other Kind and FRP Combinators

fast_join_pow(mstat, kind, n)
efficiently computes $mstat(kind ** n)$

evolve(start, next_state, n_steps=1)
evolves a system through a specified number of steps

bayes(observed_y, x, y_given_x) :: applies Bayes's rule

bin(scalar_kind, lower, width)
a Kind similar to **scalar_kind** but with values binned

Statistics Factories

Append(v) :: appends **v** to the input tuple

Cases(dict, default=None) :: converts **dict** to a statistic

Constantly(v) :: always returns **v**

ElementOf(obj) :: tests if input contained in **obj** via **[]**

Get(obj, default=None) :: looks up index in **obj**

Permute(spec) :: permutation statistic based on **spec**

Prepend :: prepends **v** to the input tuple

Proj[spec]
projection onto components based on **spec**
spec is one or more indices, slice, or int iterable
Projections are **1-indexed**, as in math not Python

Shift(m) :: cyclically shifts components right by **m**

IndexOf(v) :: returns the first index of **v** in its input, or -1 if none

Keep(cond) :: keeps input components that satisfy a condition

MaybeMap :: combines **ForEach** and **Keep** in one pass

Round(d) :: rounds a number to **d** digits

Dot(v) :: dot product of its input with **v**

Diffs(m) :: successive **m**th order differences of input

ChiSquare(expected) :: chi-squared statistic on its input

Builtin Statistics

Tuple Functions

_ :: The “hole” statistic returns its input value. **Id** is an alias.
Scalar is similar but requires a scalar result.

Count and **Dim** :: return the input's dimension

Sum and **Product** :: sum and product of input components

Abs :: root sum of squares of the input components

SumSq :: sum of squares of the input components

Negate :: negates all input components

Ascending and **Descending**
sort input components in increasing and decreasing order

Arithmetic Functions

Sqrt :: square root

Abs :: absolute value on scalars

Negate :: negates its input

Exp :: the exponential function

Log, Log2, Log10 :: logarithms of various bases

Floor :: greatest integer $\leq$ its input

Ceiling :: least integer $\geq$ its input

Trigonometric Functions

Sin, Cos, Tan :: sine, cosine, tangent functions

ASin, ACos, ATan2 :: inverse trig functions

Sinh, Cosh, Tanh :: hyperbolic trig functions

FromDegrees, FromRadians :: convert radians $\leftrightarrow$ degrees

Special Functions

NormalCDF :: CDF of the standard Normal distribution

Binomial :: Binomial coefficients

Gamma, GammaLn :: Gamma and log Gamma functions

Statistical Summaries

Min, Max :: min and max of the components

ArgMin, ArgMax :: index (0-based) of min and max component

Diff :: differences of successive input components

Mean :: mean of input components

StDev :: std deviation of input components

Variance :: variance of input components

Median :: median of input components

Quartiles :: quartiles of input components

IQR :: inter-quartile range

Bag :: creates a bag from input components

Freqs :: counts of unique components in descending order

Conditions

Distinct :: tests if all components distinct

top :: always returns true

bottom :: always returns false

Statistic Combinators

Chain(s1, s2, ..., sn)
composes statistics, s1 **then** s2 then ... then sn

Compose(s1, s2, ..., sn)
composes statistics, s1 **after** s2 after ... after sn

ForEach(stat), ForEach(stat, by=m)
applies stat successive chunks of the input tuple,
chunk size based on smallest **codim**(stat) or by

ForEachIndexed(stat), ForEachIndexed(stat, by=m)
like ForEach but passes chunk index to stat also

Fork(s1, s2, ..., sn)
joins s1(input) ... sn(input) into one tuple

IfThenElse(cond, statt, statf)
if cond true on input, apply statt, else apply statf

Tensor(s1, s2, ..., sn)
apply statistics to successive chunks of the input,
combining results into one tuple

Condition Combinators

And(cond1, cond2, ..., condn)
logical-and of all conditions on input

Or(cond1, cond2, ..., condn)
logical-or of all conditions on input

Xor(cond1, cond2, ..., condn)
logical-xor of all conditions on input

Not(cond) :: logical-not of condition

All(cond) :: true if cond true for **all** input components

Any(cond) :: true if cond true for **any** input components

Actions

unfolded(k) :: shows the unfolded Kind tree

clean(k) :: removes branches with very small weights

Market.demo
purchase batch of FRPs (of same Kind) in Marketplace

Market.buy :: purchase FRPs (of same Kind) at various prices

Market.compare
compare purchased batches of FRPs of different Kinds

Utilities

Tuples and Quantities

frplib uses a special type of tuple called **vector tuples** that add, subtract, and scale like vectors, unlike ordinary Python tuples.
frplib also uses high-precision **quantities** in values.

tup(x...) :: combines arguments into vector tuple of quantities

VecTuple.join(x...)
joins iterables and scalars into one vector tuple

v.map(f) :: vector tuple with components $f(v_i)$

as_scalar(v) :: converts a 1-tuple to a true scalar

as_quantity(x), is_quantity(x), is_quantifiable(x)
converts x to or tests if x is a quantity or convertible to
symbols, strings, numbers, nothing are quantifiable

**numeric_exp(x), numeric_ln(x), numeric_log2(x),
numeric_log10(x), numeric_abs(x), numeric_sqrt(x),
numeric_floor(x), numeric_ceil(x)**
numeric special functions that act on quantities

as_float(x) :: convert x to Python float

as_numpy(v) :: converts vector tuple to numpy array

from_numpy(x) :: converts 1-d numpy array to vector tuple

Property Accessors

dim(x) :: the dimension of 'x', if available

codim(x) :: the codimension of 'x', if available

size(x) :: the size of 'x', if available

typeof(x) :: the type of 'x' as a string

values(k) :: the **set** of a Kind's values

Symbolic Manipulation

is_symbolic(x) :: returns true if 'x' is symbolic

symbol(name) :: creates a symbolic term named by given string

symbols(names)
creates symbols with names (space-separated)

gen_symbol() :: a unique symbol name on every call

substitution(quantity, **kw)
substitutes symbols with values given in keywords

substitute(quantity, dict)
like 'substitution' but values taken from dict

substitute_with(dict)
returns a function that does **substitute(dict)**
on the quantity given to it

Function Helpers

identity :: function that returns its argument

const(a) :: a function that returns a for any input

compose(f, g) :: the function composition f after g

iterate(f, n, x0)
returns nth item in $x0, f(x0), f(f(x0)), \dots$

iterates(f, n, x0)
returns first n items from $x0, f(x0), f(f(x0)), \dots$

Sequence Helpers

irange :: creates an inclusive integer ranges with optional gaps

**index_of(v, xs, not_found=-1),
index_where(predicate, xs, not_found=-1)**
searches xs for value or true predicate

every(f, xs) :: true if $f(x)$ truthy for all x in iterable

some(f, xs) :: true if $f(x)$ truthy for some x in iterable

lmap(f, xs) :: a **list** containing $f(x)$ for every x in iterable

fold(f, init, xs), foldl(f, xs)
folds over a sequence with f from init or first element

frequencies(iterable, counts_only=False)
computes counts of unique values in iterable

Input/Output

show(x) :: display aggregate objects in a more friendly manner

add_to_search_path(paths) :: add directories to import path